

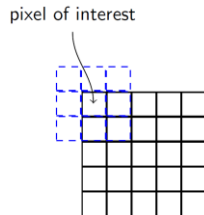
Convolutional Neural Networks

Acknowledgement: Most content of this lecture are based on **CS7015** course taught by Mitesh Khapra at IIT-Madras and Deep Learning for Computer Vision by Prof. Vineeth

Convolution Operation

- For **2D data (grayscale images)**, the convolution operation between a filter $W^{k \times k}$ and an image $X^{N_1 \times N_2}$ can be expressed as:

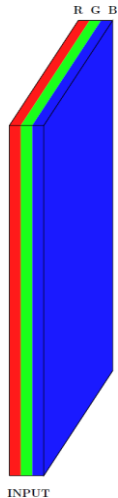
$$Y(i, j) = \sum_{u=-k}^k \sum_{v=-k}^k W(u, v) X(i - u, j - v)$$



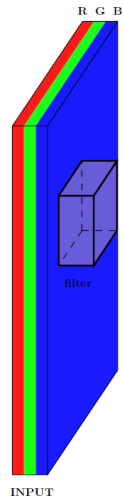
- In the **1D case**, we slide a one-dimensional filter over a one-dimensional input
- In the **2D case**, we slide a two-dimensional filter over a two-dimensional input
- What would happen in the 3D case where your images are in color (RGB)?

Convolution Operation

- What would a 3D filter look like?

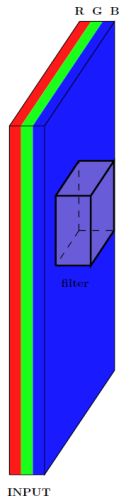


Convolution Operation

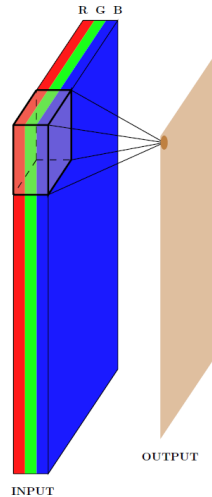


Convolution Operation

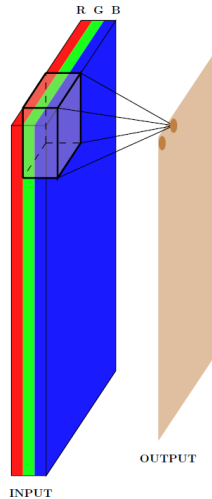
- What would a 3D filter look like?
- It will be a 3D volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



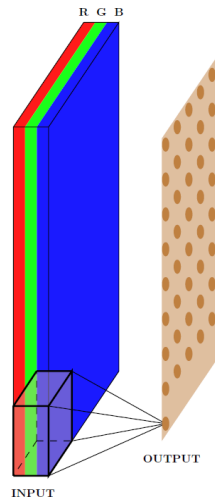
Convolution Operation



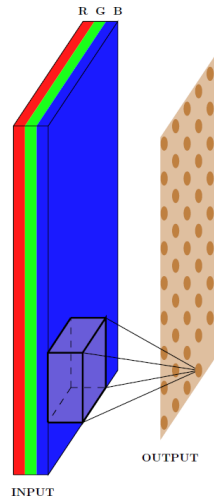
Convolution Operation



Convolution Operation

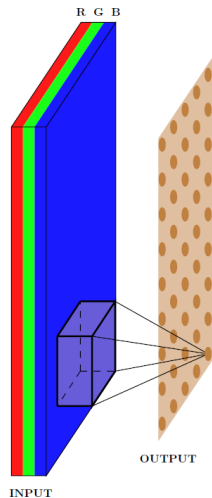


Convolution Operation



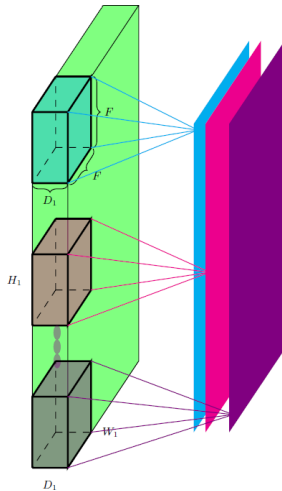
Convolution Operation

- We assume that the filter always extends to the depth of the image
- Doing a 2D convolution operation on a 3D input
- As a result the output will be 2D
- We can apply multiple filters to get multiple feature maps



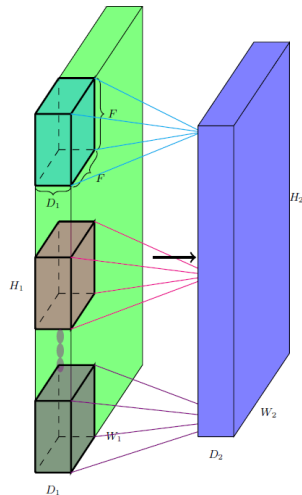
Convolution: Understanding the (Hyper)Parameters

- Input dimensions: Width (W_1) $\times$ Height (H_1) $\times$ Depth (D_1)
- Spatial extent (F) of each filter (the depth of each filter is same as the depth of input)
- Output dimensions is $W_2 \times H_2 \times D_2$
- Stride (S) (explained in following slides)
- Number of filters K



Convolution: Understanding the (Hyper)Parameters

- Input dimensions: Width (W_1) $\times$ Height (H_1) $\times$ Depth (D_1)
- Spatial extent (F) of each filter (the depth of each filter is same as the depth of input)
- Output dimensions is $W_2 \times H_2 \times D_2$
- Stride (S) (explained in following slides)
- Number of filters K



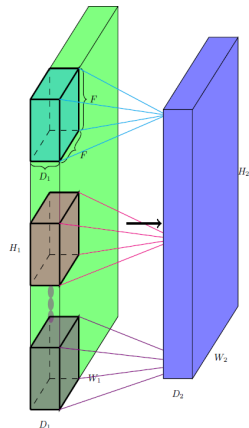
Convolution: Understanding the (Hyper)Parameters

- Input dimensions: Width (W_1) $\times$ Height (H_1) $\times$ Depth (D_1)
- Spatial extent (F) of each filter (the depth of each filter is same as the depth of input)
- Output dimensions is $W_2 \times H_2 \times D_2$
- Stride (S) (explained in following slides)
- Number of filters K

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

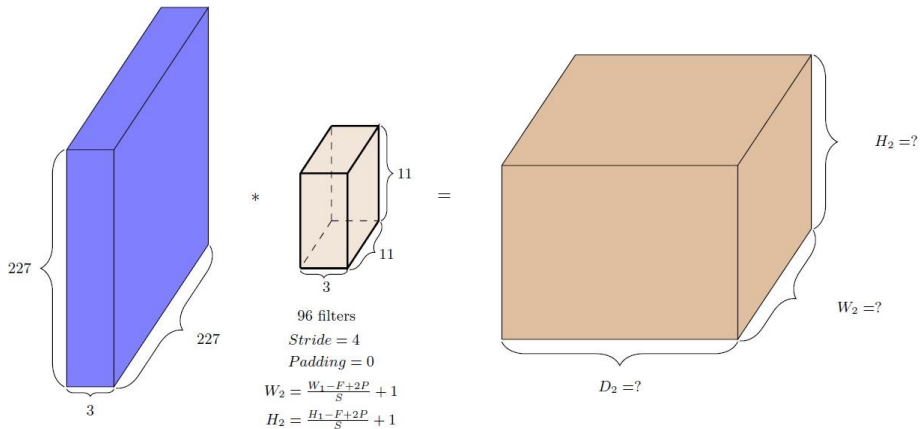
$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

$$D_2 = K$$

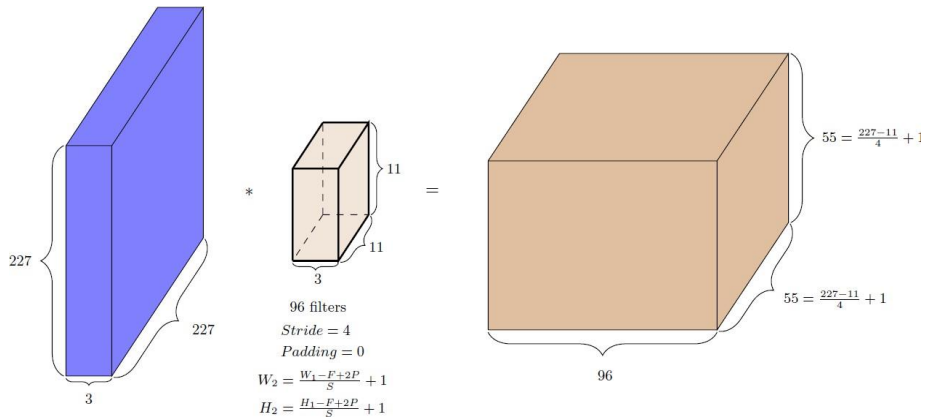


Exercise

Out dimensions for the setting below ?



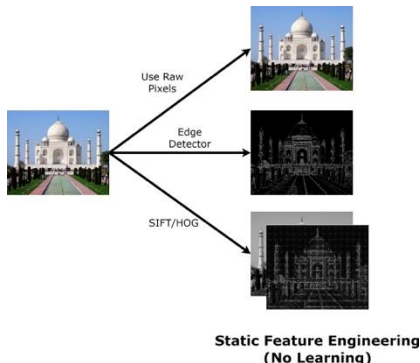
Exercise



- What is the connection between convolution and neural networks?
- Which is better ?
- We will try to understand this by considering the task of "image classification"

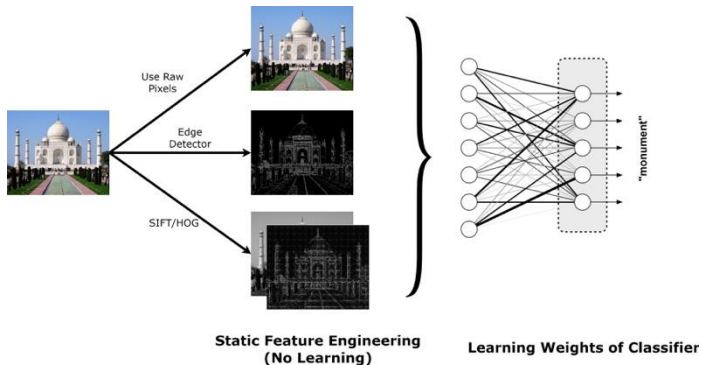
Traditional Machine Learning for Vision

- Traditional ML-based computer vision solutions involve static feature engineering from images (e.g. recall SIFT, LBP, HoG, etc)
- Though effective, static feature engineering was a bottleneck of pre-DL vision solutions



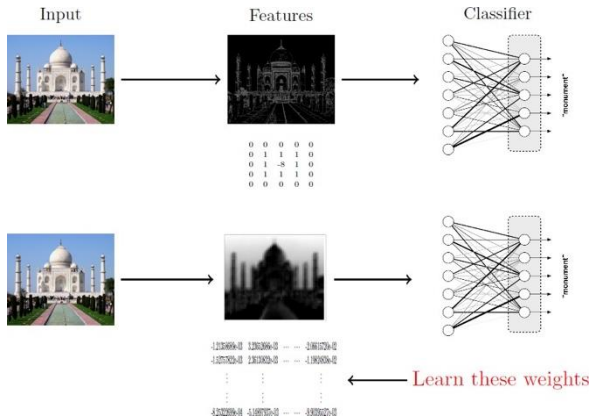
Traditional Machine Learning for Vision

- Traditional ML-based computer vision solutions involve static feature engineering from images (e.g. recall SIFT, LBP, HoG, etc)
- Though effective, static feature engineering was a bottleneck of pre-DL vision solutions



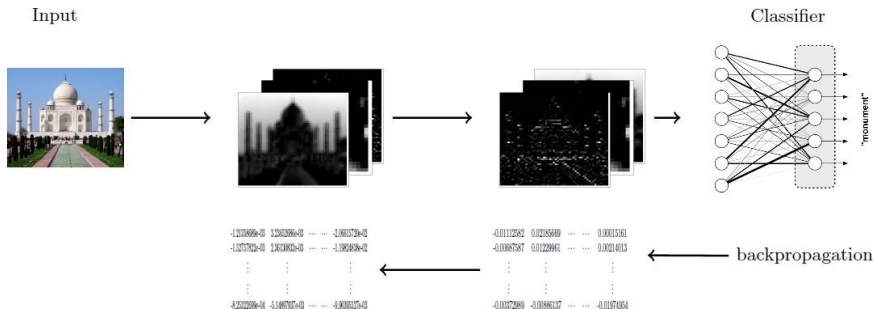
Beyond Static Feature Engineering

- Instead of using handcrafted kernels such as edge detectors can we **learn** **meaningful kernels/filters** in addition to learning the weights of the classifier?

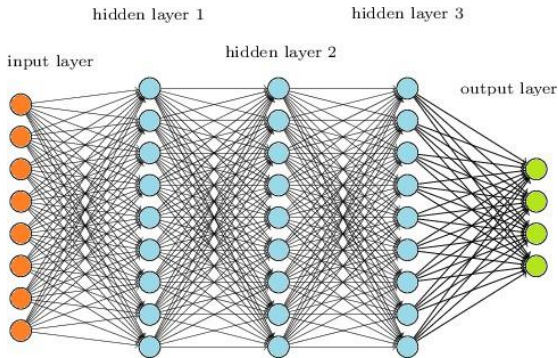


Beyond Static Feature Engineering

- Can we learn multiple **layers** of meaningful kernels/filters in addition to learning the weights of the classifier?
- Treating these kernels as parameters and learning them in addition to the weights of the classifier
- Such a network is called a Convolutional Neural Network!



- Learning kernels/filters by treating them as parameters is interesting
- But why not directly use flattened images with fully connected neural networks (FNNs) instead?



Challenges of Applying FNNs to Images



MNIST Dataset

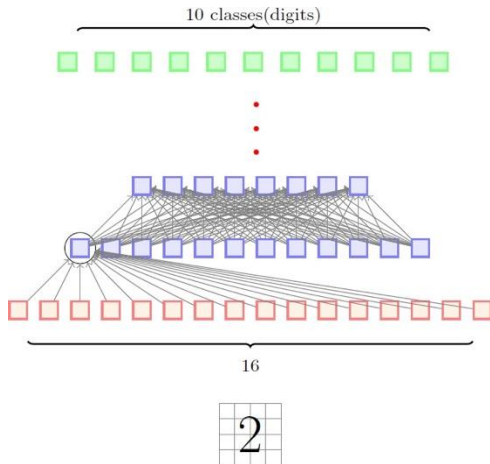
- On a reasonably *simple* dataset like MNIST, we can get about 2% error (or even better) using FNNs, but
 - Ignores spatial (2-D) structure of input images – unroll each 28×28 image into a 784-D vector
 - Pixels that are spatially separate are treated the same way as pixels that are adjacent
 - No obvious way for networks to learn same features (e.g. edges) at different places in the input image
-
- Can get computationally expensive for large images
 - For a 1MP color image with 20 neurons in the first hidden layer, how many weights in the first layer? **60 million!**

How do Convolutional Neural Networks Solve these Challenges?

- **Local receptive fields**, in which hidden units are connected to local patches of the layer below, serve two purposes:
 - Capture local spatial relationships in pixels (which would not be captured by FNNs)
 - Greatly reduces number of parameters in the model
 - For a 1MP color image a filter size of $K_1 \times K_2$ in the first hidden layer, how many weights in a convolutional layer? $K_1 \times K_2$, compare with 60 million for FNNs on the previous slide!
- **Weight sharing**, which also serves two purposes:
 - Enables translation-invariance of neural network to objects in images
 - Reduces number of parameters in the model
- **Pooling** which condenses information from previous layer, serves two purposes:
 - Aggregates information, especially minor variations
 - Reduces size of output of a previous layer, which reduces number of computations in later layers

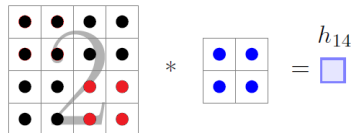
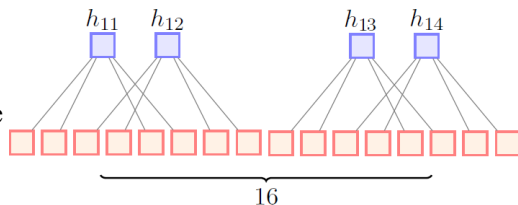
Local Receptive Fields

- This is what a regular feedforward neural network will look like
- There are many dense connections here
- All 16 input neurons are contributing to computation of h_{11}
- What happens to this in case of convolution



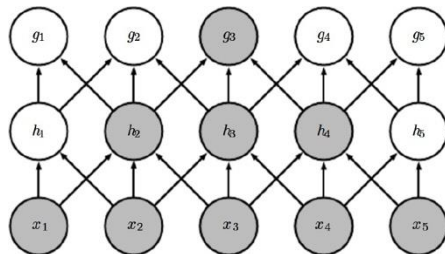
Local Receptive Fields

- Only a few local neurons participate in computation of h_{11}
- E.g. only pixels 1, 2, 5, 6 contribute to h_{11}
- The connections are much sparser $\rightarrow$ reduces the number of parameters in the model
- Takes advantage of the structure of the image (interactions between neighboring pixels are interesting in images)



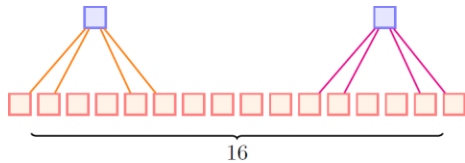
Local Receptive Fields

- But is sparse connectivity really a good thing?
- Aren't we losing information (by losing interactions between some input pixels)
- **Not really**
- The two highlighted neurons (x_1, x_5) do not interact in layer 1
- But they indirectly contribute to the computation of g_3 and hence interact indirectly



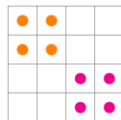
Weight Sharing

- Consider the following network;
 - do we want the kernel weights to be different for different parts of the image?



● Kernel 1

● Kernel 2



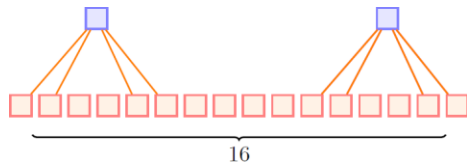
4x4 Image

Weight Sharing

- Consider the following network;
 - do we want the kernel weights to be different for different parts of the image?

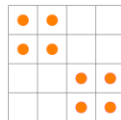
Not really.

- We want the filter to respond to an object or an artefact in an image in the same way irrespective of where it is located in the image $\Rightarrow$ **translation-invariance**



● Kernel 1

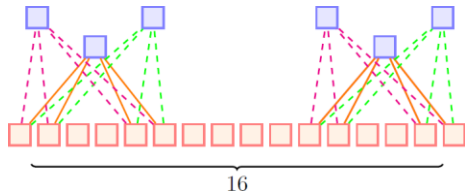
● Kernel 2



4x4 Image

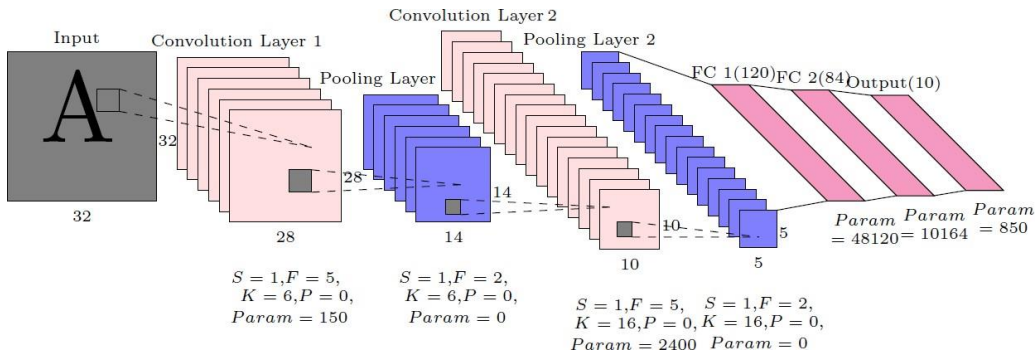
Weight Sharing

- We can have as many different kernels to capture different kinds of artifacts,
- but each one is intended to give the the same response on all parts of the image
- This is called **weight sharing**



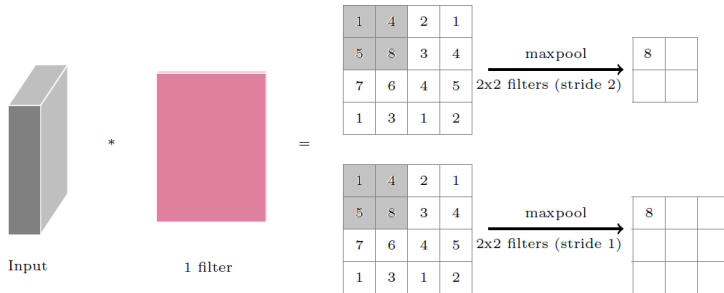
Convolutional Neural Network

A typical CNN looks as follows:



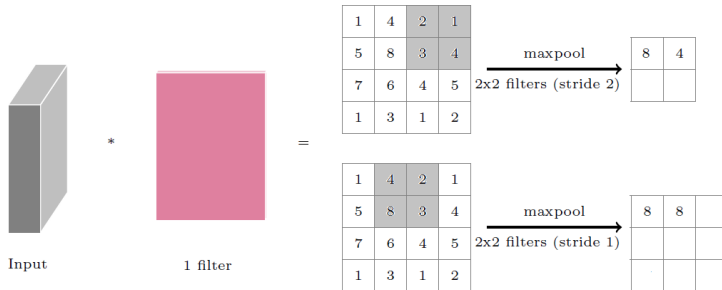
- It has alternate convolution and pooling layers
- What do pooling layers do?

Pooling Layer



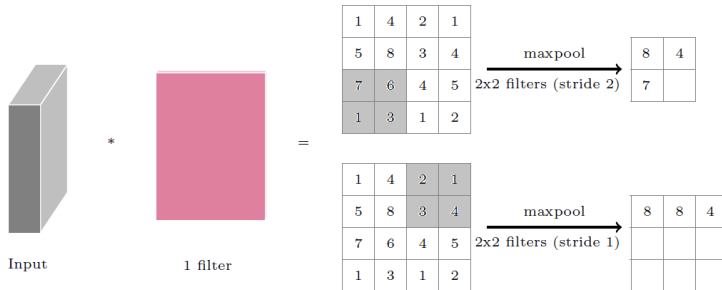
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



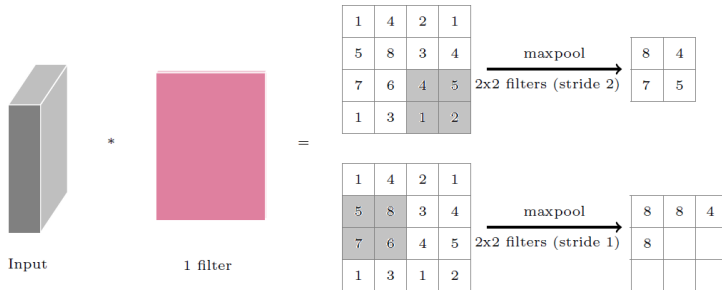
- Pooling** is a parameter-free down sampling operation

Pooling Layer



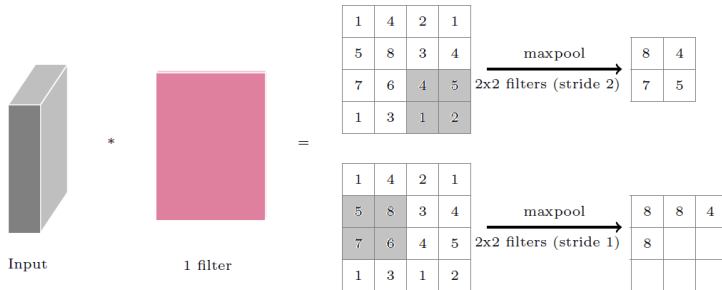
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



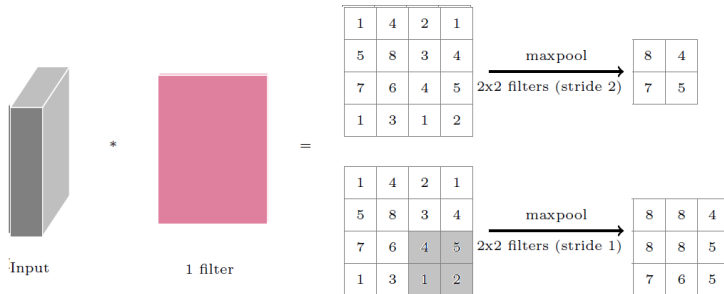
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



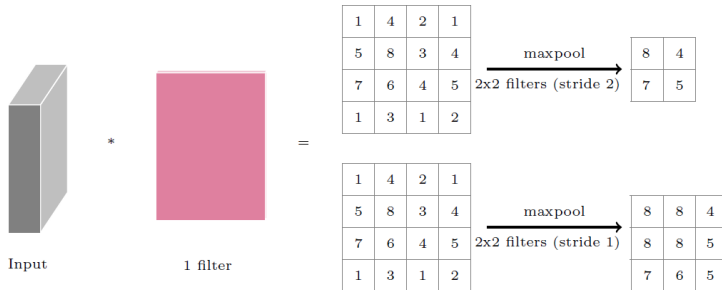
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



- **Pooling** is a parameter-free down sampling operation

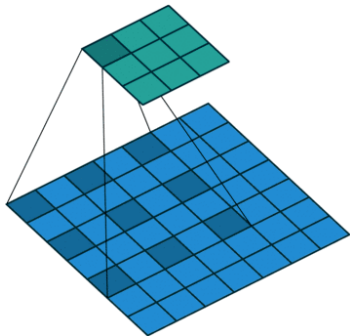
Pooling Layer



- **Pooling** is a parameter-free down sampling operation
- Instead of **Max Pooling**, we can also do **Average Pooling**, etc
- Others: Mixed Pooling (combines max and average pooling), Spatial Pyramid Pooling, Spectral Pooling

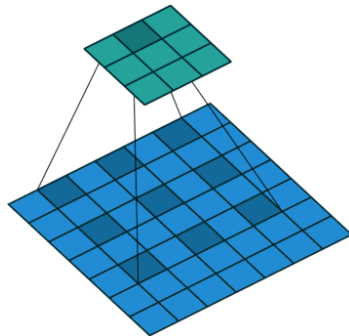
Other Variants of Convolution: Dilated Convolution

- Introduces another parameter to convolutional layer called **dilation rate**
- Controls spacing between values in a kernel
- Figure shows 3×3 kernel with dilation rate 2



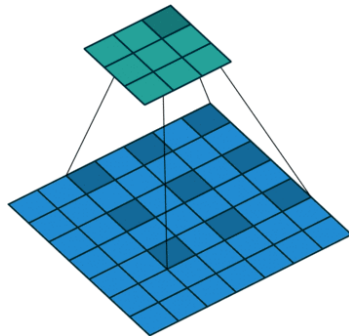
Other Variants of Convolution: Dilated Convolution

- Introduces another parameter to convolutional layer called **dilation rate**
- Controls spacing between values in a kernel
- Figure shows 3×3 kernel with dilation rate 2



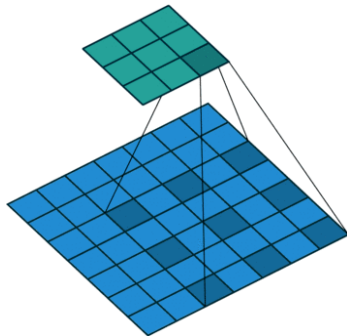
Other Variants of Convolution: Dilated Convolution

- Introduces another parameter to convolutional layer called **dilation rate**
- Controls spacing between values in a kernel
- Figure shows 3×3 kernel with dilation rate 2

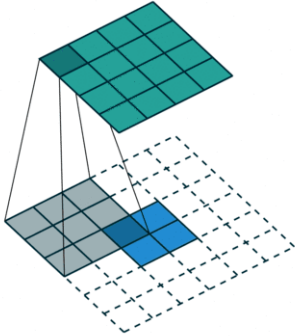


Other Variants of Convolution: Dilated Convolution

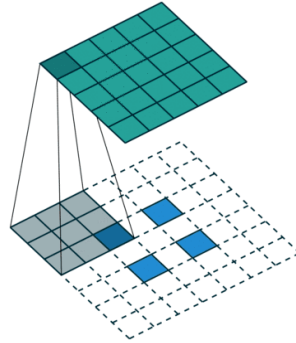
- Introduces another parameter to convolutional layer called **dilation rate**
- Controls spacing between values in a kernel
- Figure shows 3×3 kernel with dilation rate 2
- Notice that dilated rate 1 is standard convolution



Other Variants of Convolution: Transpose Convolution

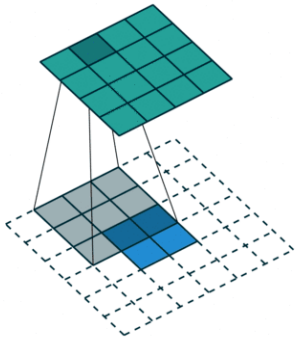


Upsampling 2×2 input to a 4×4 output

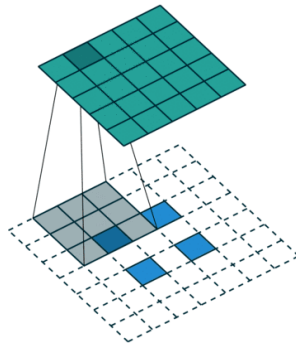


Upsampling 2×2 input to a 5×5 output

Other Variants of Convolution: Transpose Convolution

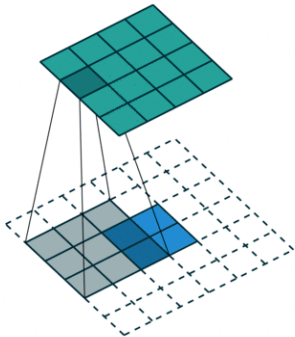


Upsampling 2×2 input to a 4×4 output

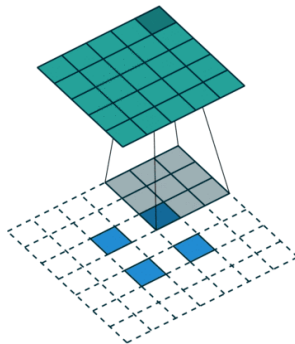


Upsampling 2×2 input to a 5×5 output

Other Variants of Convolution: Transpose Convolution

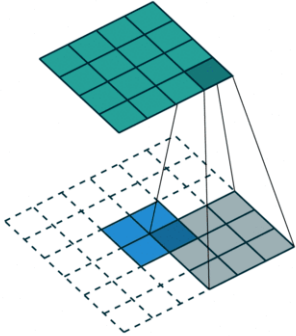


Upsampling 2×2 input to a 4×4 output

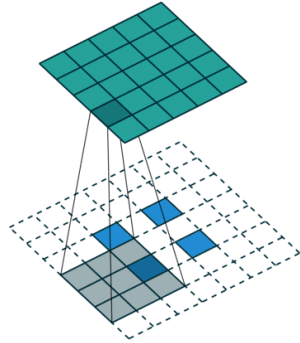


Upsampling 2×2 input to a 5×5 output

Other Variants of Convolution: Transpose Convolution

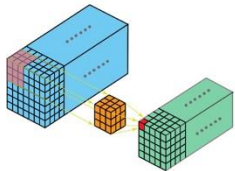


Upsampling 2×2 input to a 4×4 output

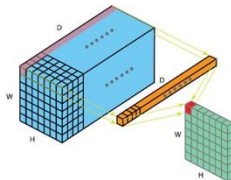


Upsampling 2×2 input to a 5×5 output

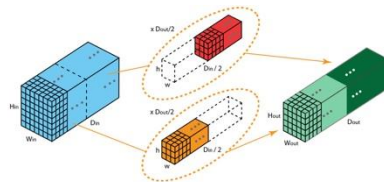
Other Variants of Convolution



3D Convolution



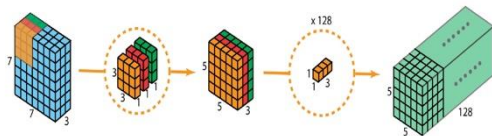
1×1 Convolution
Pointwise Convolution



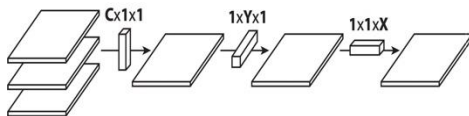
Grouped Convolution

Credit: Illarion Khlestov, Chi-Feng Wang

Other Variants of Convolution



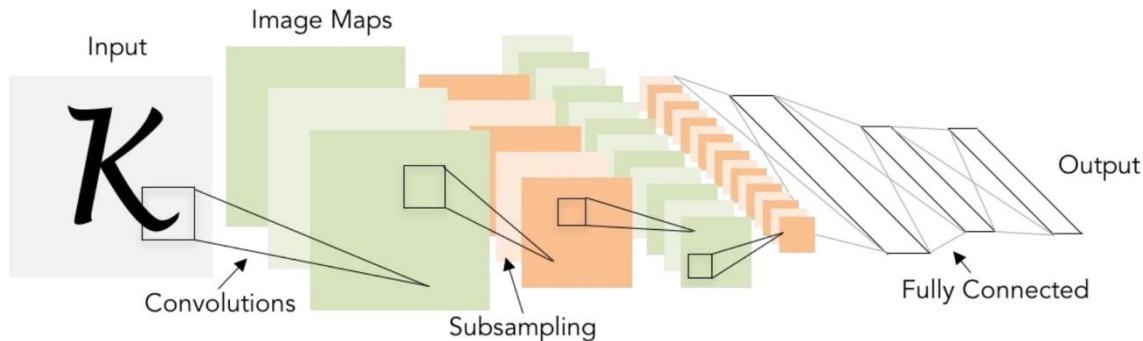
Depthwise Separable Convolution



Flattened Convolutions

Evolution of CNN Architectures for Image Classification

LeNet-5 (1989-1998)



- By Yann LeCun and team for handwritten digits classification
- Conv filters were 5×5 , applied at stride 1
- Subsampling (Pooling) layers were 2×2 applied at stride 2
- **Overall Architecture:** [CONV-POOL-CONV-POOL-FC-FC]

ImageNet Classification Challenge

- Image database organized according to WordNet hierarchy (currently only nouns)
- Currently, over five hundred images per node
- Started the ImageNet LSVRC in 2010, for benchmarking of methods for image classification
- Performance measure in Top-1 error and Top-5 error
- <http://www.image-net.org/>



Loss Functions: Beyond Mean Square Error

- **Cross-Entropy Loss Function:** Most popular for classification
- Given by:

$$\begin{aligned} L &= -\frac{1}{C} \sum_{i=1}^C y_i \log \hat{y}_i \\ &= -y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \text{ (binary case)} \end{aligned}$$

- When activation function is sigmoid ($\sigma(x) = \frac{1}{1+e^{-x}}$), derivative of cross-entropy loss function, $\frac{\partial L}{\partial w_j}$, w.r.t. a weight in last layer, w_j , is:

$$\begin{aligned} \frac{\partial L}{\partial w_j} &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \frac{\partial \sigma}{\partial w_j} \\ &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \sigma'(z) x_j \\ &= \frac{1}{n} \sum_x \frac{\sigma'(z) x_j}{\sigma(z)(1-\sigma(z))} (\sigma(z) - y) \\ &= \frac{1}{n} \sum_x x_j (\sigma(z) - y) \end{aligned}$$

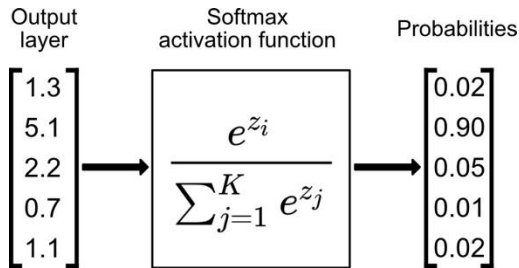
Note the last term in the final expression, very similar to gradient of MSE loss function

Activation Function in Output Layer

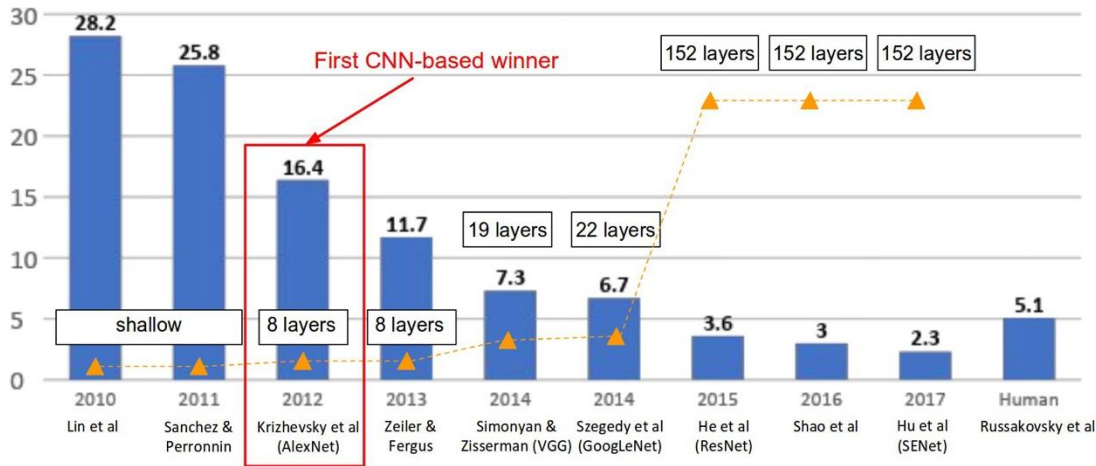
Softmax activation function

$$a_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

Helps convert output layer values (also called **logits**) to probability scores

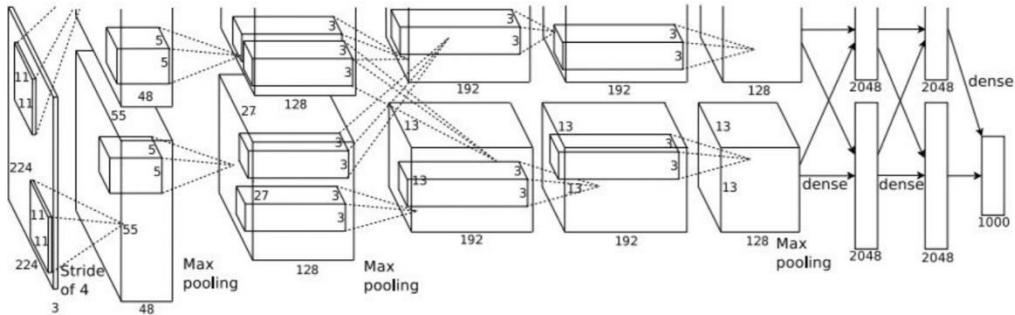


Winners of ImageNet Classification Challenge



Credit: Fei-Fei Li, Justin Johnson and Serena Yeung, CS231n course, Stanford, Spring 2019

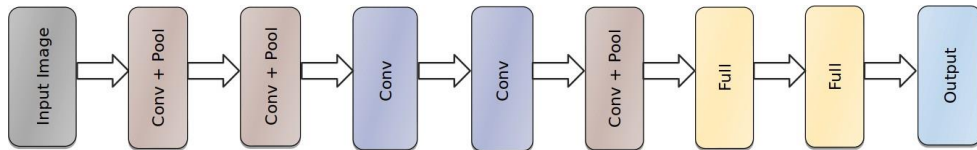
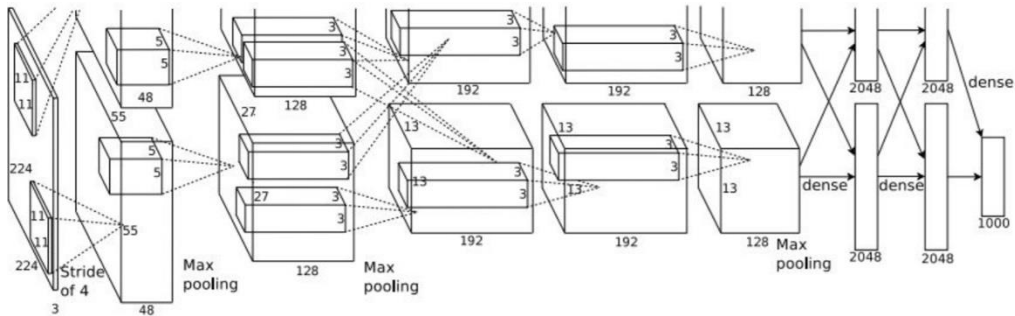
AlexNet¹



- Winner of ImageNet LSVRC-2012
- Overall architecture design similar to LeNet; but deeper with conv layers stacked on top of each other
- Trained over 1.2M images using SGD with regularization

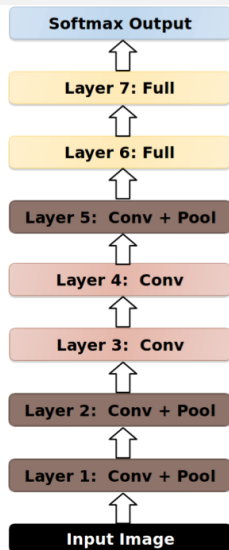
¹Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." NIPS 2012.

AlexNet

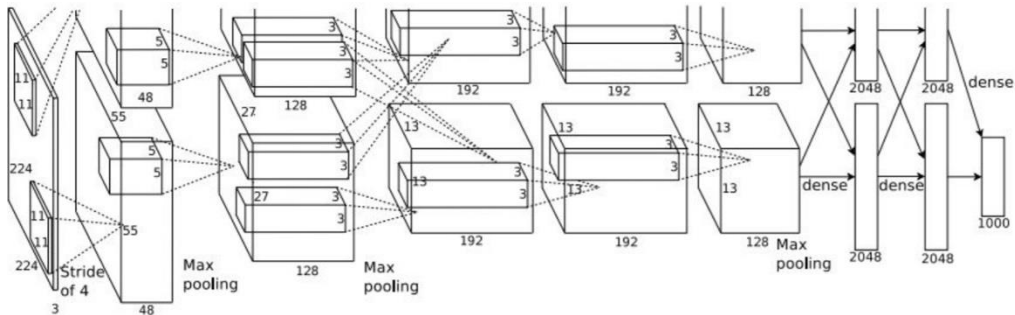


AlexNet

- 8 layers in total (5 convolutional layers, 3 fully connected layers)
- Trained on ImageNet Dataset
- **Response normalization layers** follow the first and second convolutional layers.
- Max-pooling follow first, second and the fifth convolutional layers
- The ReLU non-linearity is applied to the output of every layer



AlexNet

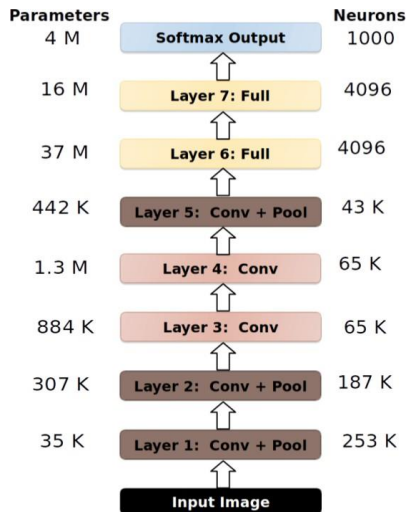


About 57 M parameters are in the fully connected layers

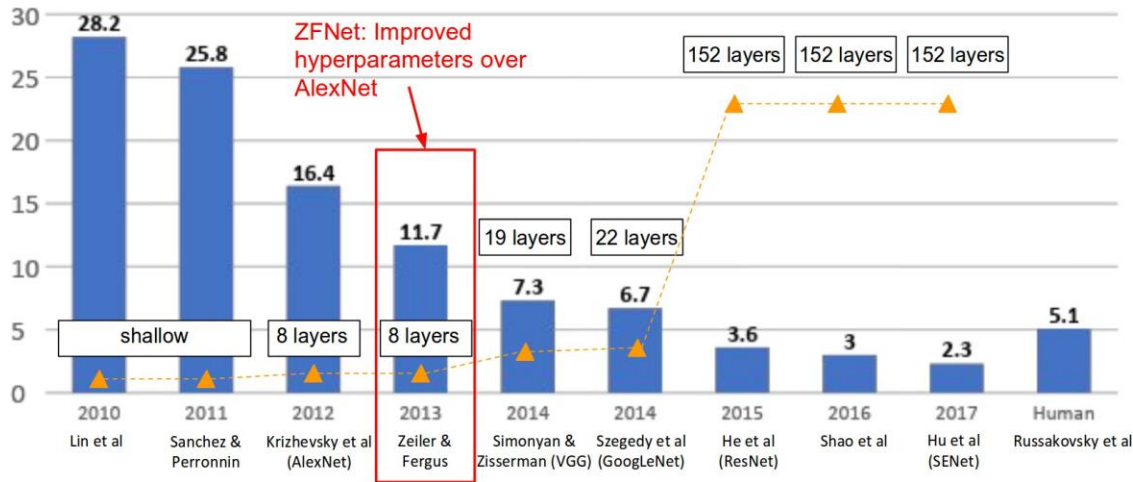
									Total
Parameters :	$[(11 \times 11 \times 3) + 1] \times 96 = 35 \text{ K}$	$[5 \times 5 \times 48] \times 256 = 307 \text{ K}$	$[3 \times 3 \times 256] \times 384 = 884 \text{ K}$	663 K	442 K	37 M	16 M	4 M	60 M
Neurons :	253,440	$27 \times 27 \times 256 = 186,624$	$13 \times 13 \times 384 = 64,896$	$13 \times 13 \times 384 = 64,896$	$13 \times 13 \times 256 = 43,264$	4096	4096	1000	0.63 M

AlexNet

- Convolutional layers cumulatively contain about 90 – 95% of computation, only about 5% of the parameters
- Fully-connected layers contain about 95% of parameters.
- Trained with SGD, on two NVIDIA GTX 580 3GB GPUs for about a week

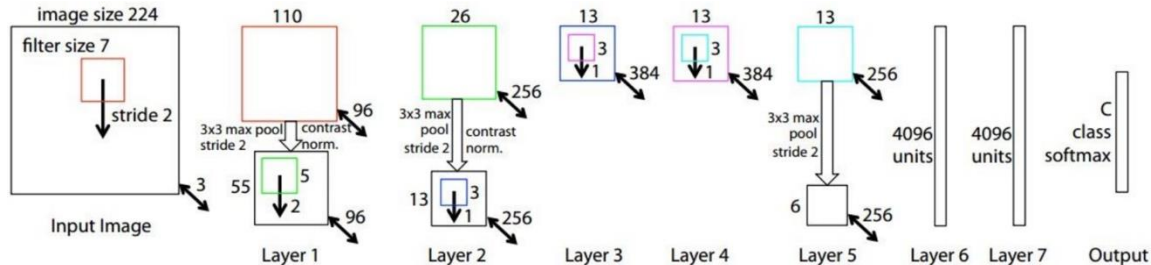


Winners of ImageNet Classification Challenge



Credit: Fei-Fei Li, Justin Johnson and Serena Yeung, CS231n course, Stanford, Spring 2019

ZFNet (2013)



Similar to AlexNet but:

CONV1: change from $(11 \times 11 \text{ stride } 4)$ to $(7 \times 7 \text{ stride } 2)$

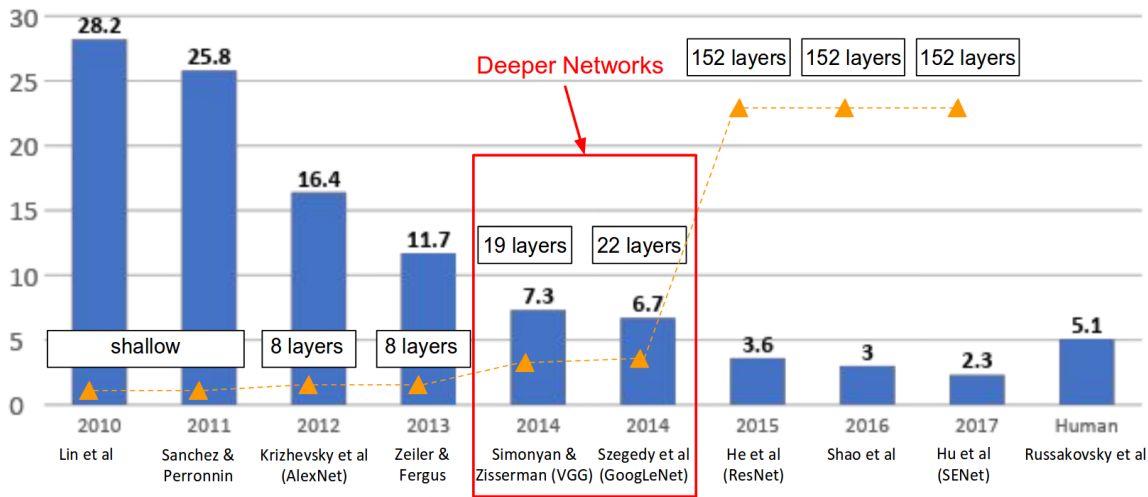
CONV3,4,5: instead of 384, 384, 256 filters, use 512, 1024, 512

ImageNet top-5 error: 16.4% $\rightarrow$ 11.7%

Credit: Fei-Fei Li, Justin Johnson and Serena Yeung, CS231n course, Stanford, Spring 2019

²Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014

Winners of ImageNet Classification Challenge



Credit: Fei-Fei Li, Justin Johnson and Serena Yeung, CS231n course, Stanford, Spring 2019

VGG Net

- Runner-up in ILSVRC 2014
- More layers lead to more nonlinearities
- **Key contribution:** Depth of the network is a critical component for good performance
- **Homogeneous Architecture:** From beginning to end:
 - 3 x 3 CONV stride 1 pad 1
 - 2 x 2 MAX POOL stride 2
- **Smaller receptive fields:** less parameters; faster
 - two 3 x 3 conv has same receptive field as a single 5 x 5 conv;
 - three 3 x 3 conv has same receptive field as a single 7 x 7 conv
- Fewer parameters: $3 \times 3^2 C^2$ (vs) $7^2 C^2$



VGG Net

- VGG19 only slightly better than VGG16
- Used ensembles of networks for best results
- Uses a lot more memory and parameters



11-layer



13-layer



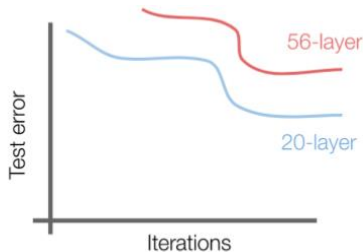
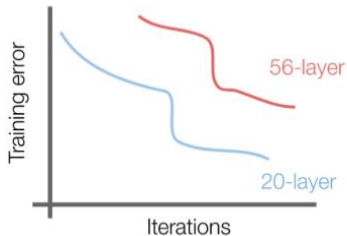
16-layer



19-layer

How Deeper Can we go ?

- What happens when we continue stacking deeper layers on a *plain* CNN?



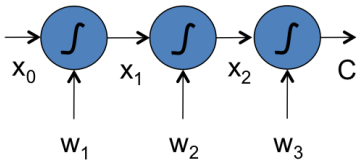
- Deeper model does worse than shallow model!

Why?

- The initial guess is that the deep model is overfitting since it is much bigger than the shallow model
- The deep model is actually underfitting since it also performs worse than the shallow model on the training set

Vanishing/Exploding gradient

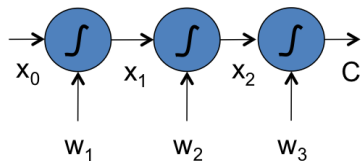
- Consider a network


$$\frac{\partial L}{\partial x_0} = \overset{< 1/4}{\sigma'(w_3^T x_2)} \times w_3 \times \overset{< 1/4}{\sigma'(w_2^T x_1)} \times w_2 \times \overset{< 1/4}{\sigma'(w_1^T x_0)} \times w_1$$

Why?

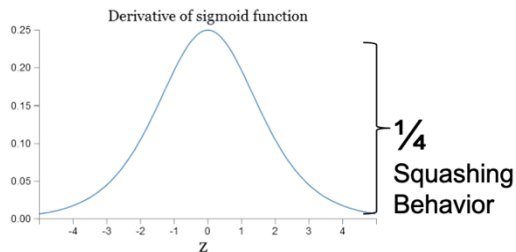
Vanishing/Exploding gradient

- Consider a network



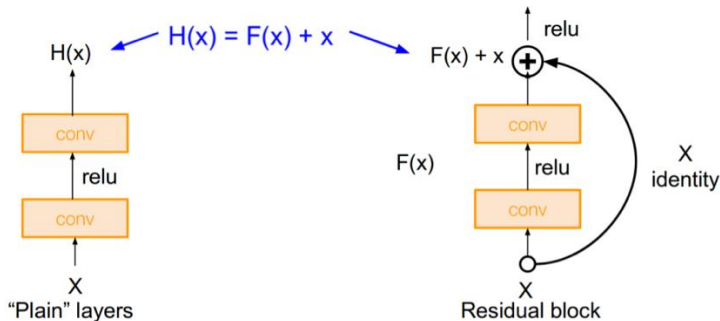
$$\frac{\partial L}{\partial x_0} = \overset{< 1/4}{\sigma'(w_3^T x_2)} \times w_3 \times \overset{< 1/4}{\sigma'(w_2^T x_1)} \times w_2 \times \overset{< 1/4}{\sigma'(w_1^T x_0)} \times w_1$$

- Vanishing gradients:** Deeper the network, gradients vanish quickly, thereby slowing the rate of change in initial layers
- Exploding gradients:** Happen when the individual layer gradients are much higher than 1, for instance - can be overcome by gradient clipping



ResNet

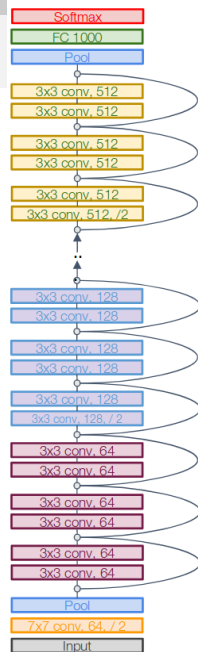
- The deeper model should be able to perform at least as well as the shallower model
- Solution: Change the network with identity connections between layers:



- Use network layers to fit residual $F(x) = H(x) - x$ instead of $H(x)$ directly

ResNet (residual network)

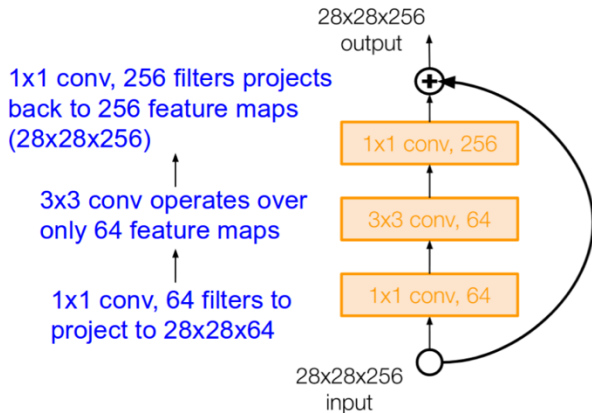
- A stack of many residual blocks
- Each residual block has two 3x3 conv layers
- Periodically, double number of filters and down sample spatially using stride 2 ($/2$ in each dimension)
- Use global average pooling and a single linear layer at the end (FC 1000 to output classes)
- Total depths of 34, 50, 101, or 152 layers for ImageNet dataset



Credit: Fei-Fei Li, Justin Johnson and Serena Yeung, CS231n course, Stanford, Spring 2019

ResNet

- For deeper networks (ResNet-50+), use bottleneck layer to improve efficiency



ResNet

- Able to train very deep networks
- Deeper networks performs better than shallow networks now (as expected); residual blocks help avoid vanishing gradient
- 1st place in all ILSVRC and COCO 2015 competitions

ResNet @ ILSVRC & COCO 2015 Competitions

1st place in all five major challenges

- ImageNet Classification: "Ultra-deep" 152-layer nets
- ImageNet Detection: 16% better than the 2nd best
- ImageNet Localization: 27% better than the 2nd best
- COCO Detection: 11% better than the 2nd best
- COCO Segmentation: 12% better than the 2nd best

References

- Please refer to Mitesh Khapra's original lecture slides (and videos) for more detailed explanation of some of these topics, available at [CS7015: Deep Learning](#).
- Also refer to Vineet Balasubramanian original lecture slide and videos for more detailed information [CS5370: Deep Learning for Vision](#)

Other good resources:

[Deep Learning Book:](#)

Stanford [CS231n Notes](#)